OPERATING SYSTEMS – ASSIGNMENT 2 (CO3)

Q1. Processes and Process States

1. Introduction

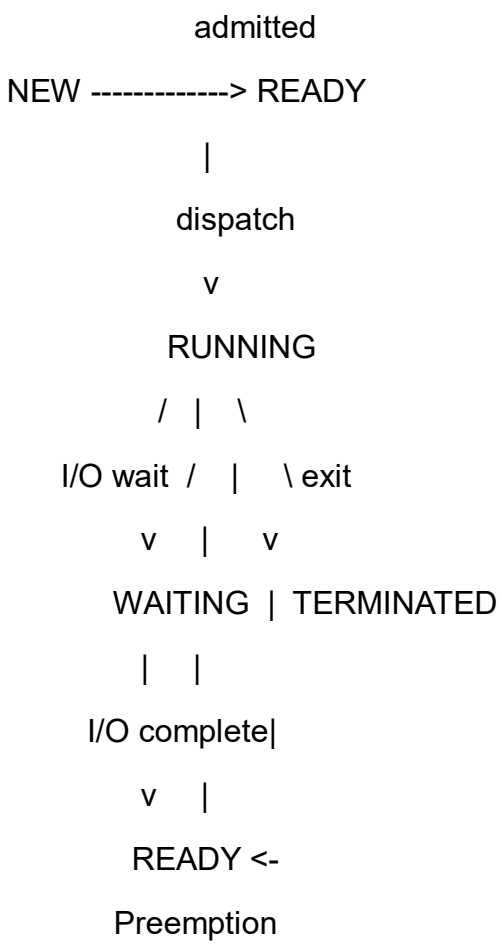
A process is a program currently under execution. In an online examination server, separate processes may handle student sessions, answer saving, and video monitoring. When thousands of students log in simultaneously, the operating system must manage many active processes efficiently.

2. Process States

A process passes through five main states:

1. New: The process is being created.
2. Ready: The process is waiting for CPU time.
3. Running: The CPU is executing the process.
4. Waiting: The process is waiting for an I/O operation or event.
5. Terminated: The process has completed execution.

Process-state diagram:



3. Process Control Block (PCB)

A Process Control Block is a data structure maintained by the operating system to store information about a process.

PCB component	Description
Process ID	Unique process identification number
Process state	Current state of the process
Program counter	Address of the next instruction
CPU registers	Stores register values
CPU scheduling information	Priority and scheduling details
Memory information	Memory allocation details
I/O status	Open files and I/O devices
Accounting information	CPU time and process usage

4. Process Creation Using fork()

The fork() system call creates a new child process. The child process receives a separate process ID and executes independently.

C program:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>


int main()

{

    pid_t pid = fork();


    if (pid < 0)

        printf("Process creation failed");


    else if (pid == 0)

        printf("Child PID: %d\n", getpid());
```

```
    else {
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        wait(NULL);
    }

    return 0;
}
```

Sample output:

Parent PID: 2500

Child PID: 2501

5. Observing Processes in Linux

The following commands help observe running processes:

ps -ef

ps -f

ps -p PID

cat /proc/PID/status

6. Performance Evaluation for 2,000 Processes

When 2,000 processes run simultaneously, the following factors affect performance:

- Context switching: The CPU saves one process's state and loads another process's state. Frequent switching consumes CPU time.
- Memory consumption: Every process requires its own virtual address space and process-management information.
- I/O waiting: Processes waiting for disk, network, or other I/O operations cannot use the CPU during that waiting period.
- Scheduling overhead: The operating system must select processes from the ready queue.
- Resource competition: Processes compete for CPU, memory, files, and network resources.
- **7. Comparison of Process Designs**

Design	Advantages	Disadvantages
One process per student	Strong isolation and fault protection	High memory usage
One thread per student	Lower memory overhead and faster communication	Shared-memory errors can affect other sessions
Fixed worker-process pool	Controls resource usage and supports parallel processing	Requires a task distribution mechanism

8. Alternative: Thread-Based Design

Instead of creating one process for every student, the server can create a thread for each session or use a fixed thread pool.

Threads share the process's code, data, and open files. Each thread has its own stack and CPU register state.

A thread generally requires less memory than a separate process because it does not need a separate complete address space. Thread switching can also be less expensive than process switching.

However, a serious error in a shared process can affect multiple sessions.

9. Final Recommendation

A fixed pool of worker processes or threads is suitable for the examination server. It limits resource consumption and avoids creating thousands of independent processes. Critical services such as answer saving should also use reliable storage and proper synchronisation.

Conclusion: Process pools provide a practical balance between performance, memory usage, and reliability for a large online examination server.

Q2. CPU Scheduling

1. Introduction

CPU scheduling is the method used by the operating system to select a process from the ready queue and allocate CPU time to it.

A cloud video-encoding service handles both short and long jobs. Short jobs should not wait unnecessarily behind long jobs. Therefore, an efficient scheduling algorithm is required.

2. Scheduling Algorithms

1. First Come First Serve (FCFS)

- Processes execute in arrival order.
- It is simple to implement.
- It is non-preemptive.
- Long processes may make short processes wait.

2. Shortest Job First (SJF)

- The process with the shortest burst time executes first.
- It reduces average waiting time when burst times are known.
- Long processes may suffer starvation.

3. Shortest Remaining Time First (SRTF)

- It is the preemptive version of SJF.
- A newly arrived process can replace the currently running process if its remaining burst time is shorter.
- It provides good response for short jobs.

4. Priority Scheduling

- Each process receives a priority.
- The highest-priority process executes first.
- Low-priority processes may suffer starvation.
- Ageing gradually increases the priority of waiting processes.

5. Round Robin (RR)

- Every process receives a fixed time quantum.
- If a process is not completed, it returns to the ready queue.

- It provides fair CPU sharing.

3. Example Process Table

The assignment provides the following processes for its Round Robin program:

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

For the Round Robin example, the time quantum is 3.

4. Important Scheduling Formulas

Turnaround Time:

$$ TAT = Completion\ Time - Arrival\ Time $$

Waiting Time:

$$ WT = Turnaround\ Time - Burst\ Time $$

Response Time:

$$ RT = First\ CPU\ Start\ Time - Arrival\ Time $$

Average Waiting Time:

$$ AWT = \frac{\sum WT}{Number\ of\ Processes} $$

Average Turnaround Time:

$$ ATAT = \frac{\sum TAT}{Number\ of\ Processes} $$

5. Round Robin Gantt Chart

For quantum = 3:

0 3 6 9 12 15 16 19 21 23 26

| P1 | P2 | P3 | P4 | P1 | P2 | P3 | P4 | P1 | P3 |

6. Round Robin Calculation

Process	Completion time	Turnaround time	Waiting time
P1	23	23	15
P2	16	15	11

P3	26	24	15
P4	21	18	13

$$Average\ TAT = \frac{23+15+24+18}{4}=20$$

$$Average\ WT = \frac{15+11+15+13}{4}=13.5$$

7. Round Robin C Program:

```
#include <stdio.h>

#define N 4

int main()
{
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int left[N], finish[N];
    int queue[64], added[N] = {0};
    int head = 0, tail = 0;
    int time = 0, done = 0, quantum = 3;

    for (int i = 0; i < N; i++)
        left[i] = burst[i];

    queue[tail++] = 0;
    added[0] = 1;

    printf("Gantt:");

    while (done < N) {
        int p = queue[head++];
        int run = left[p] < quantum ? left[p] : quantum;
```

```
    printf(" P%d(%d-%d)", p + 1, time, time + run);

    time += run;
    left[p] -= run;

    for (int i = 0; i < N; i++) {
        if (!added[i] && arrival[i] <= time) {
            queue[tail++] = i;
            added[i] = 1;
        }
    }

    if (left[p] > 0)
        queue[tail++] = p;
    else {
        finish[p] = time;
        done++;
    }
}

double tw = 0, tt = 0;

printf("\nProcess TAT WT\n");

for (int i = 0; i < N; i++) {
    int tat = finish[i] - arrival[i];
    int wt = tat - burst[i];
```

```
        tt += tat;
        tw += wt;

        printf("P%d      %d  %d\n", i + 1, tat, wt);
    }

    printf("Average TAT = %.2f\n", tt / N);
    printf("Average WT = %.2f\n", tw / N);

    return 0;
}
```

8. Effect of Time Quantum

Quantum	Behaviour
1	More context switches; good sharing but higher overhead
3	Balanced CPU sharing
5	Fewer context switches
20	Behaves more like FCFS for these jobs

A very small quantum increases context-switch overhead. A very large quantum reduces responsiveness because long jobs can hold the CPU for longer periods.

9. Starvation and Ageing

Starvation occurs when a process waits for a very long time because other processes are continuously selected.

Ageing solves this problem by gradually increasing the priority of waiting processes.

For example, a process waiting for a long time can receive a higher priority until it gets CPU time.

10. Multilevel Feedback Queue

A Multilevel Feedback Queue uses multiple queues with different priorities and time quanta.

- New jobs enter a high-priority queue.
- Short jobs may finish quickly in the first queue.

- CPU-intensive jobs move to lower-priority queues.
- Waiting jobs can receive priority increases through ageing.

11. Final Recommendation

Round Robin or Multilevel Feedback Queue is suitable for the video-encoding service. Round Robin provides fairness, while a Multilevel Feedback Queue gives faster service to short jobs and allows long jobs to continue in lower-priority queues.

Q3. Threads

1. Introduction

A thread is the smallest unit of CPU execution within a process. A process can contain multiple threads that execute different tasks concurrently.

In a traffic-camera number-plate recognition system, 64 cameras generate image frames. Processing them one after another may cause delays. Multithreading allows several camera frames to be processed concurrently.

2. Resources Shared by Threads

Threads belonging to the same process share:

- Program code
- Global variables
- Heap memory
- Open files
- Process resources

Each thread has its own:

- Stack
- Program counter
- CPU registers
- Thread ID

3. Types of Threads

User-Level Threads

- Managed by a user-level thread library.
- Fast to create and manage.
- The kernel may not know about individual threads.
- A blocking system call can block the entire process in some many-to-one implementations.

Kernel-Level Threads

- Managed directly by the operating system.
- Can execute on multiple CPU cores.
- A blocked thread does not necessarily block other threads.

- Creation and management have greater overhead.

4. Thread Mapping Models

Model	Description	Advantage	Disadvantage
Many-to-One	Many user threads map to one kernel thread	Low overhead	No true parallel execution
One-to-One	Each user thread maps to one kernel thread	Supports parallelism	Higher overhead
Many-to-Many	Many user threads map to several kernel threads	Flexible and scalable	More complex management

5. POSIX Thread Program

```
#include <stdio.h>
#include <pthread.h>

#define N 8

void *process_camera(void *arg)
{
    int id = *(int *)arg;
    printf("Camera %d processed\n", id);
    return NULL;
}

int main()
{
    pthread_t t[N];
    int id[N];

    for (int i = 0; i < N; i++) {
        id[i] = i + 1;
```

```
        pthread_create(&t[i], NULL, process_camera, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    printf("All cameras processed\n");
    return 0;
}
```

Sample output:

```
Camera 1 processed
Camera 2 processed
Camera 3 processed
Camera 4 processed
Camera 5 processed
Camera 6 processed
Camera 7 processed
Camera 8 processed
All cameras processed
```

6. Thread Pool Comparison

Thread count	Expected behaviour
4 threads	Lower resource usage, but more waiting
8 threads	Better parallel processing
16 threads	More concurrent work, but increased scheduling overhead
64 threads	One thread per camera, but potentially high overhead

The best thread-pool size depends on the number of CPU cores, image-processing workload, and I/O activity.

7. Speed-up Calculation

$$ Speedup = \frac{Serial\ Execution\ Time}{Parallel\ Execution\ Time} $$

For example, if serial execution takes 64 seconds and parallel execution takes 16 seconds:

\[Speedup = \frac{64}{16}=4 \]

This is an illustrative calculation, not a measured result.

Speed-up stops increasing because of:

- Limited CPU cores
- Thread creation overhead
- Shared-resource contention
- Memory bandwidth limitations
- Synchronisation delays

8. Race Condition

A race condition occurs when multiple threads access and modify shared data simultaneously without proper synchronisation.

For example, if two threads increment the same counter, one update may overwrite the other.

9. Alternative: Multiple Processes

Using several processes instead of threads provides separate address spaces.

Feature	Threads	Processes
Memory	Shared address space	Separate address spaces
Creation	Generally faster	Generally slower
Communication	Shared variables	IPC mechanisms
Isolation	Lower	Higher
Resource usage	Generally lower	Generally higher

10. Final Recommendation

A fixed thread pool of 8 or 16 threads can be tested for the traffic-camera system. The final number should be selected using actual execution-time measurements and CPU utilisation.

Q4. Inter-Process Communication (IPC)

1. Introduction

Inter-Process Communication is a mechanism that allows two or more processes to exchange information and coordinate their activities.

In a weather station, one process collects sensor readings while another process analyses the readings. IPC allows these processes to exchange data continuously.

2. Pipe Communication

A pipe is a communication channel that transfers data between processes. It is commonly used between related processes, such as a parent and child.

Working:

1. **The parent creates a pipe using pipe().**
2. **The parent creates a child using fork().**
3. **The parent writes sensor readings into the pipe.**
4. **The child reads the readings from the pipe.**
5. **The parent closes its write end after sending all data.**
6. **The child receives zero from read() when the pipe reaches end-of-file.**

3. Pipe Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];

    if (pipe(fd) == -1)
        return 1;

    pid_t pid = fork();
```

```
if (pid == 0) {
    close(fd[1]);

    int r, count = 0, sum = 0, max = 0;

    while (read(fd[0], &r, sizeof(r)) > 0) {
        count++;
        sum += r;

        if (r > max)
            max = r;
    }

    printf("analyser: %d readings, average %.1f, max %d\n",
           count, count ? (double)sum / count : 0.0, max);

    close(fd[0]);
    return 0;
}

close(fd[0]);

int readings[] = {31, 33, 36, 34, 41, 38};

for (int i = 0; i < 6; i++)
    write(fd[1], &readings[i], sizeof(readings[i]));

close(fd[1]);
```

```
    wait(NULL);

    printf("reader: done\n");

    return 0;
}
```

4. Why Must the Write End Be Closed?

The child continues reading until the pipe returns zero, which indicates that all write ends have been closed.

If the parent does not close fd[1], the child may wait indefinitely because the pipe is still open for writing. The child cannot know that no more readings will arrive.

5. Shared Memory

Shared memory allows two or more processes to access the same memory region.

Working:

1. **The operating system creates a shared memory region.**
2. **Both processes attach the region.**
3. **The producer writes sensor readings.**
4. **The consumer reads the readings.**
5. **Synchronisation controls access to shared data.**

6. Comparison of IPC Methods

Feature	Pipe	Shared memory	Message queue
Communication	Byte stream	Common memory region	Messages
Data copying	Kernel buffering and copying	Direct access after mapping	Kernel-managed message transfer
Synchronisation	Basic pipe ordering	Requires extra synchronisation	Message operations provide queue coordination

Speed	**Suitable for simple communication**	**Often faster for large data**	**Depends on message size**
Complexity	**Simple**	**More complex**	**Moderate**

7. Data Transfer Investigation

The assignment asks for measurements using 1 MB, 10 MB, and 100 MB.

A suitable measurement table is:

Data size	Pipe time	Shared-memory time
1 MB	**Record measured time**	**Record measured time**
10 MB	**Record measured time**	**Record measured time**
100 MB	**Record measured time**	**Record measured time**

Actual values must be collected by running the programs on the Linux system. They depend on hardware, kernel, and implementation.

8. Alternative: Shared Memory with a Semaphore

A shared-memory design can use a semaphore to indicate when new data is available.

- **The producer writes a reading into shared memory.**
- **The producer signals the semaphore.**
- **The consumer waits on the semaphore.**
- **The consumer reads the available reading.**
- **The processes repeat this operation.**

Shared memory avoids repeatedly transferring the data through a pipe, but it requires careful synchronisation to prevent reading incomplete or overwritten data.

9. Complexity Analysis

For transferring \(n\) readings, both methods must handle all \(n\) readings, so their basic data-processing work is \(O(n)\).

- **Pipe: incurs kernel buffering and data-copying overhead.**
- **Shared memory: avoids repeated pipe data copying but requires synchronisation.**
- **Message queue: provides structured message exchange but has kernel-management overhead.**

10. Final Recommendation

For a small weather-station data stream, a pipe is simple and reliable. For large continuous data transfers, shared memory with proper synchronisation can provide better performance.

Q5. Synchronisation: Critical Sections, Semaphores and Monitors

1. Introduction

Synchronisation controls the execution of multiple processes or threads that access shared resources.

In a railway ticket-booking system, thousands of agents may try to book the same seat. Without synchronisation, two agents may receive the same seat.

2. Critical Section

A critical section is the part of a program where shared data or resources are accessed.

A correct critical-section solution must satisfy three conditions:

1. **Mutual Exclusion: Only one process can enter the critical section at a time.**
2. **Progress: If the critical section is free, the selection of the next process should not be delayed unnecessarily.**
3. **Bounded Waiting: Every process requesting entry must receive an opportunity within a limited number of turns.**

3. Peterson's Solution

Peterson's algorithm is a software solution to the critical-section problem for two processes.

It uses:

- **flag[2] to indicate whether a process wants to enter.**
- **turn to decide which process gets priority.**

Advantages:

- **Provides mutual exclusion for two processes.**
- **Satisfies progress.**
- **Provides bounded waiting under its assumptions.**

Limitations:

- **Designed for two processes.**
- **Depends on assumptions about memory ordering.**
- **Modern processors and compilers may reorder memory operations. Suitable atomic operations and memory-ordering guarantees are needed for a correct modern implementation.**

4. Semaphores

A semaphore is a synchronisation variable used to control access to shared resources.

Two main operations are:

- **wait(): Decreases the semaphore value or blocks the process if the resource is unavailable.**
- **signal(): Increases the semaphore value and may wake a waiting process.**

Types:

Type	Description
Binary semaphore	Has values 0 and 1; used for exclusive access
Counting semaphore	Controls access to multiple instances of a resource

5. Producer-Consumer Problem

The producer creates data and places it in a shared buffer. The consumer removes data from the buffer.

Three synchronisation variables are used:

- **empty: Number of empty buffer slots.**
- **full: Number of occupied buffer slots.**
- **mutex: Protects the buffer during updates.**

6. Producer-Consumer C Program

```
#include <stdio.h>

#include <pthread.h>

#include <semaphore.h>


#define SIZE 5

#define ITEMS 20


int buffer[SIZE], in = 0, out = 0;

sem_t empty, full;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```
long consumed_sum = 0;

void *producer(void *arg)
{
    (void)arg;

    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);

        buffer[in] = item;
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    (void)arg;

    for (int k = 0; k < ITEMS; k++) {
        sem_wait(&full);
        pthread_mutex_lock(&lock);

        int item = buffer[out];
```

```
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&empty);

        consumed_sum += item;
    }

    return NULL;
}

int main()
{
    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_t p, c;

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS, consumed_sum, ITEMS * (ITEMS + 1) / 2);

    sem_destroy(&empty);
    sem_destroy(&full);
```

```
    return 0;
}
```

7. Readers-Writers Problem

The readers-writers problem involves multiple readers and writers accessing shared data.

- **Multiple readers can read simultaneously.**
- **Only one writer can modify the data at a time.**
- **A writer must have exclusive access.**
- **Synchronisation prevents inconsistent data.**

8. Dining Philosophers Problem

The dining philosophers problem demonstrates resource sharing and deadlock.

Five philosophers sit around a table. Each needs two forks to eat.

If all philosophers pick up one fork and wait for another, deadlock may occur.

Solutions:

- **Allow only four philosophers to compete simultaneously.**
- **Use a waiter to control fork allocation.**
- **Assign forks a fixed ordering.**
- **Use semaphores to coordinate access.**

9. Monitors

A monitor is a high-level synchronisation mechanism that combines shared data and operations into one protected structure.

Only one thread can execute inside a monitor at a time.

Condition variables are used to wait for a condition and signal another waiting thread.

Advantages:

- **Simplifies synchronisation.**
- **Reduces accidental simultaneous access.**
- **Provides structured access to shared data.**

- **Makes complex programs easier to maintain.**

10. Locking Strategies for Railway Booking

Locking method	Advantages	Disadvantages
One global lock	Simple and safe	Low concurrency
One lock per train	Different trains can be booked simultaneously	Agents booking the same train must wait
One lock per coach	Higher concurrency	More complex lock management

A lock per coach may provide high concurrency, but operations involving multiple coaches must acquire locks carefully to avoid deadlock.

Q6. Deadlock Handling and Banker's Algorithm

1. Introduction

Deadlock occurs when a group of processes waits indefinitely for resources held by one another.

In a research centre, processes may require GPUs, software licences, and high-speed storage. Incorrect resource allocation can stop the entire system.

2. Four Necessary Conditions for Deadlock

1. Mutual Exclusion

At least one resource must be non-shareable.

2. Hold and Wait

A process holds one resource while waiting for another.

3. No Preemption

Resources cannot be forcibly taken from a process.

4. Circular Wait

A circular chain exists in which every process waits for a resource held by the next process.

All four conditions must exist simultaneously for a deadlock to occur.

3. Resource-Allocation Graph

A resource-allocation graph represents processes and resources.

- **Circle: Process.**
- **Square: Resource.**
- **Process to resource: Request edge.**
- **Resource to process: Assignment edge.**

4. Banker's Algorithm

The Banker's algorithm is a deadlock-avoidance algorithm.

It checks whether granting a resource request will leave the system in a safe state.

Important data structures:

- **Available: Number of available resources.**
- **Maximum: Maximum demand of each process.**

- **Allocation: Resources currently allocated.**
- **Need: Remaining resource requirement.**

\[Need = Maximum - Allocation \]

5. Given Resource Data

Available:

\[Available = (3,3,2) \]

Maximum matrix:

Process	A	B	C
P0	7	5	3
P1	3	2	2
P2	9	0	2
P3	2	2	2
P4	4	3	3

Allocation matrix:

Process	A	B	C
P0	0	1	0
P1	2	0	0
P2	3	0	2
P3	2	1	1
P4	0	0	2

Need matrix:

Process	A	B	C
P0	7	4	3
P1	1	2	2
P2	6	0	0
P3	0	1	1

P4 4 3 1

6. Safe Sequence

Initial available resources:

\[Work=(3,3,2) \]

- **P1 can finish because Need = (1,2,2). New Work = (5,3,2).**
- **P3 can finish because Need = (0,1,1). New Work = (7,4,3).**
- **P4 can finish because Need = (4,3,1). New Work = (7,4,5).**
- **P0 can finish because Need = (7,4,3). New Work = (7,5,5).**
- **P2 can finish because Need = (6,0,0). New Work = (10,5,7).**

Therefore, the safe sequence is:

\[P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2 \]

7. Completed Banker's Algorithm Program

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R] = {3, 3, 2};

int maxm[P][R] = {
   {7, 5, 3}, {3, 2, 2}, {9, 0, 2},
   {2, 2, 2}, {4, 3, 3}
};

int alloc[P][R] = {
   {0, 1, 0}, {2, 0, 0}, {3, 0, 2},
   {2, 1, 1}, {0, 0, 2}
};
```

```
int is_safe(int seq[P])
{
    int work[R], finish[P] = {0}, count = 0;

    memcpy(work, avail, sizeof work);

    while (count < P) {
        int found = 0;

        for (int p = 0; p < P; p++) {
            if (finish[p])
                continue;

            int possible = 1;

            for (int r = 0; r < R; r++) {
                int need = maxm[p][r] - alloc[p][r];

                if (need > work[r])
                    possible = 0;
            }

            if (possible) {
                for (int r = 0; r < R; r++)
                    work[r] += alloc[p][r];

                finish[p] = 1;
                seq[count++] = p;
```

```
            found = 1;
        }
    }

    if (!found)
        return 0;
  }

  return 1;
}

void request(int p, int req[R])
{
  for (int r = 0; r < R; r++) {
    if (req[r] > maxm[p][r] - alloc[p][r] ||
        req[r] > avail[r]) {
        printf("P%d request: must wait (not available)\n", p);
        return;
    }
  }

  for (int r = 0; r < R; r++) {
    avail[r] -= req[r];
    alloc[p][r] += req[r];
  }

  int seq[P];

  if (is_safe(seq)) {
```

```
        printf("P%d request: granted, safe sequence", p);

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);

        printf("\n");
    } else {
        for (int r = 0; r < R; r++) {
            avail[r] += req[r];
            alloc[p][r] -= req[r];
        }

        printf("P%d request: denied (unsafe)\n", p);
    }
}

int main()
{
    int seq[P];

    if (is_safe(seq)) {
        printf("initial state safe:");

        for (int i = 0; i < P; i++)
            printf(" P%d", seq[i]);

        printf("\n");
    }
```

```
    request(1, (int[R]){1, 0, 2});
    request(4, (int[R]){3, 3, 0});
    request(0, (int[R]){0, 2, 0});

    return 0;
}
```

8. Why Is P0's Request Unsafe?

P0 requests:

$$ Request=(0,2,0) $$

After temporarily granting the request:

$$ Available=(3,1,2) $$

P0's remaining Need becomes:

$$ Need=(7,2,3) $$

The remaining resource requirements cannot be satisfied by the available resources and the allocations that can be safely released. The safety algorithm cannot find a complete sequence for all processes.

Therefore, the request is denied and the original allocation is restored.

9. Comparison of Deadlock Strategies

Strategy	Explanation	Advantage	Disadvantage
Prevention	Removes one necessary deadlock condition	Prevents deadlock	May reduce resource utilisation
Avoidance	Checks whether allocation is safe	Avoids unsafe states	Requires maximum demand information
Detection and recovery	Detects deadlock and recovers	Flexible allocation	Recovery may lose work
Ostrich approach	Ignores rare deadlocks	Low overhead	Requires manual recovery

10. Time Complexity

For n processes and m resource types, a typical Banker safety check has complexity:

\[O(mn^2) \]

The algorithm repeatedly checks process requirements and updates available resources until it finds a safe sequence or determines that the state is unsafe.

11. Measurement Table

The assignment asks for measurements with 10, 100, and 1,000 processes.

Number of processes	Measured time
10	Record actual execution time
100	Record actual execution time
1,000	Record actual execution time

The expected trend is an increase in execution time as the number of processes increases. Actual timings depend on the computer and implementation.

12. Final Recommendation

The research centre should use Banker's algorithm if maximum resource demands can be estimated in advance and the number of processes is manageable.

For workloads where maximum demands are unpredictable, deadlock detection and recovery may be more practical. Resource reservations and careful scheduling can further reduce deadlock risk.